

# EECE.4821/5821 COMPUTER ARCHITECTURE AND DESIGN

**INSTRUCTOR: MICHAEL BOEWE**

---

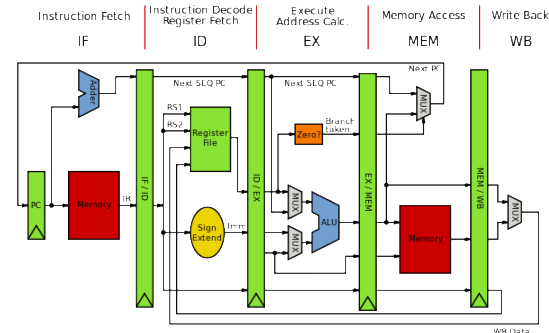
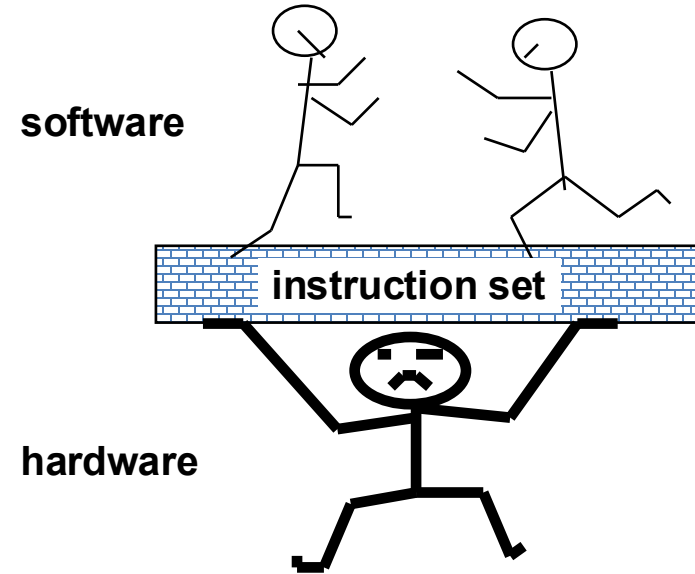
INSTRUCTIONS: LANGUAGE OF THE COMPUTER

[Adapted from *Computer Organization and Design RISC-V Edition, 2nd Edition*, Patterson & Hennessy, © 2020, MK]

[Slides copied shamelessly (but with permission) from Prof. Seung Woo Son]

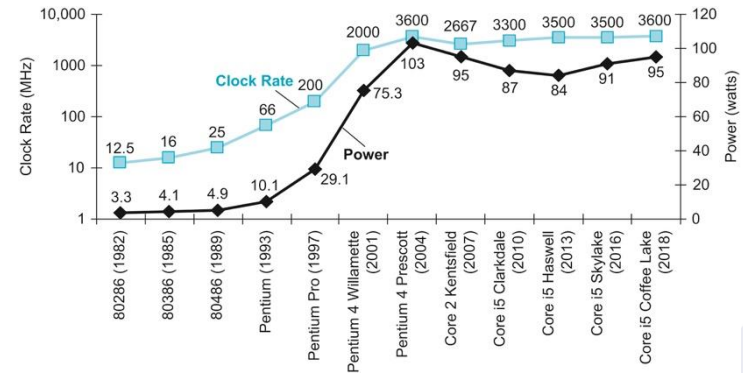
# WHAT IS COMPUTER ARCHITECTURE?

- Classical view: instruction set architecture (ISA)
  - Boundary between hardware and software
  - Provides abstraction at both high level and low level
- More modern view: ISA + hardware design
  - Can talk about processor architecture, system architecture



# REVIEW: LECTURE 1

- Cost/performance is improving
  - Due to underlying technology advancement
- Hierarchical layers of abstraction
  - In both hardware and software
- Instruction set architecture
  - The hardware/software interface
- Execution time: the best performance measure
- Power is a limiting factor
  - Use parallelism to improve performance



## REVIEW: MEASURING CPU PERFORMANCE

- A program is running on CPU A with the following parameters:
  - Total instruction count: 10M
  - Average CPI: 2.5
  - CPU clock rate: 1 GHz ( $\Rightarrow$  clock cycles? )
- What's the performance (or execution time)?
- What if the same program is used with a new compiler?
  - Changes due to compiler: IC = 5M, CPI = 3.0
  - New performance?
  - What is the speedup with this changes?

# INSTRUCTION SET

- The repertoire of instructions of a computer
- Different computers have different instruction sets
  - But with many aspects in common
- Early computers had very simple instruction sets
  - Simplified implementation
- Many modern computers also have simple instruction sets

# THE RISC-V INSTRUCTION SET

- Used as the example throughout the book
- Developed at UC Berkeley as an open ISA
- Now managed by the RISC-V Foundation ([riscv.org](https://riscv.org))
- Typical of many modern ISAs
  - See RISC-V Reference Data
- Similar ISAs have a large share of the embedded core market
  - Applications in consumer electronics, network/storage equipment, cameras, printers, ...

<https://en.wikipedia.org/wiki/RISC-V>

# RISC-V SIMULATORS OR INTERPRETERS

- QEMU
  - <https://www.qemu.org/>
- Ripes
  - <http://ripes.me/Ripes/>
  - Recommended simulator
- Spike
  - <https://github.com/riscv-software-src/riscv-isa-sim>
- Some online interpreters
  - <https://www.cs.cornell.edu/courses/cs3410/2019sp/riscv/interpreter/>
  - <https://ascslab.org/research/briscv/simulator/simulator.html>

# ARITHMETIC OPERATIONS

- Add and subtract, three operands
  - Two sources and one destination

add a, b, c // a gets b + c
- All arithmetic operations have this form
- *Design Principle 1: Simplicity favors regularity*
  - Regularity makes implementation simpler
  - Simplicity enables higher performance at lower cost

## ARITHMETIC EXAMPLE

- C code:

$f = (g + h) - (i + j);$

- Compiled RISC-V code:

```
add t0, g, h    // temp t0 = g + h
add t1, i, j    // temp t1 = i + j
sub f, t0, t1   // f = t0 - t1
```

## REGISTER OPERANDS

- Arithmetic instructions use register operands
- RISC-V has a  $32 \times 32$ -bit register file
  - Use for frequently accessed data
  - 32-bit data called a “word”
    - 32x32-bit general purpose registers x0 to x31
  - 64-bit data is called a “doubleword”
- *Design Principle 2: Smaller is faster*
  - c.f. main memory: millions of locations

## RISC-V REGISTERS

- x0: the constant value 0
- x1: return address
- x2: stack pointer
- x3: global pointer
- x4: thread pointer
- x5 – x7, x28 – x31: temporaries
- x8: frame pointer
- x9, x18 – x27: saved registers
- x10 – x11: function arguments/results
- x12 – x17: function arguments

## REGISTER OPERAND EXAMPLE

- C code:

```
f = (g + h) - (i + j);
```

— f, ..., j in x19, x20, ..., x23

- Compiled RISC-V code:

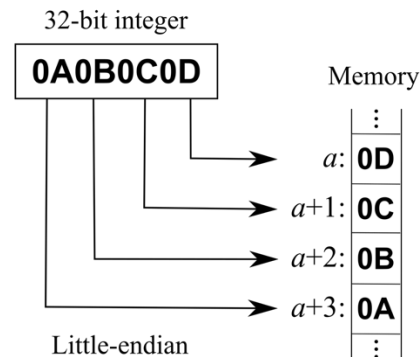
```
add x5, x20, x21
```

```
add x6, x22, x23
```

```
sub x19, x5, x6
```

# MEMORY OPERANDS

- Main memory used for composite data
  - Arrays, structures, dynamic data
- To apply arithmetic operations
  - Load values from memory into registers
  - Store result from register to memory
- Memory is byte addressed
  - Each address identifies an 8-bit byte
- RISC-V is Little Endian
  - Least-significant byte at least address of a word
  - *c.f.* Big Endian: most-significant byte at least address
- RISC-V does not require words to be aligned in memory
  - Unlike some other ISAs



## MEMORY OPERAND EXAMPLE

- C code:

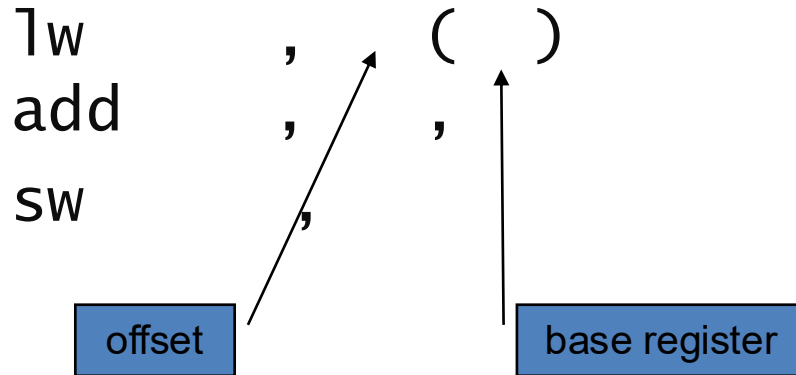
$A[12] = h + A[8];$

—  $h$  in  $x21$ , base address of  $A$  in  $x22$

- Compiled RISC-V code:

— Index 8 requires an offset of 32

- 4 bytes per word



## MEMORY OPERAND EXAMPLE

- C code:

`A[12] = h + A[8];`

- `h` in `x21`, base address of `A` in `x22`

- Compiled RISC-V code:

- Index 8 requires an offset of 32

- 4 bytes per word

```
lw    x9, 32(x22)
```

```
add   x9, x21, x9
```

```
sw    x9, 48(x22)
```

offset

base register

# REGISTERS VS. MEMORY



- Registers are faster to access than memory
- Operating on memory data requires loads and stores
  - More instructions to be executed
- Compiler must use registers for variables as much as possible
  - Only spill to memory for less frequently used variables
  - Register optimization is important!

## IMMEDIATE OPERANDS

- Constant data specified in an instruction

`addi x22, x22, 4`

- **Make the common case fast**
  - Small constants are common
  - Immediate operand avoids a load instruction

# UNSIGNED BINARY INTEGERS

- Given an n-bit number

$$x = x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \dots + x_12^1 + x_02^0$$

- Range: 0 to  $+2^n - 1$
- Example
  - 0000 0000 0000 0000 0000 0000 0000 1011<sub>2</sub>  
=  $0 + \dots + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$   
=  $0 + \dots + 8 + 0 + 2 + 1 = 11_{10}$
- Using 32 bits: 0 to +4,294,967,295
- Using 64 bits: 0 to +18,446,774,073,709,551,615

## 2S-COMPLEMENT SIGNED INTEGERS

- Given an n-bit number

$$x = -x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \dots + x_12^1 + x_02^0$$

- Range:  $-2^{n-1}$  to  $+2^{n-1}-1$
- Example
  - $- 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1100_2$   
 $= -1 \times 2^{31} + 1 \times 2^{30} + \dots + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0$   
 $= -2,147,483,648 + 2,147,483,644 = -4_{10}$
- Using 32 bits:  $-2,147,483,648$  to  $2,147,483,647$
- Using 64 bits:  $-9,223,372,036,854,775,808$  to  $9,223,372,036,854,775,807$

## 2S-COMPLEMENT SIGNED INTEGERS

- Bit 31 is sign bit
  - 1 for negative numbers
  - 0 for non-negative numbers
- $-(-2^{n-1})$  can't be represented
- Non-negative numbers have the same unsigned and 2s-complement representation
- Some specific numbers
  - 0: 0000 0000 ... 0000
  - -1: 1111 1111 ... 1111
  - Most-negative: 1000 0000 ... 0000
  - Most-positive: 0111 1111 ... 1111

# SIGNED NEGATION

- Complement and add 1
  - Complement means  $1 \rightarrow 0, 0 \rightarrow 1$

$$x + \bar{x} = 1111 \dots 111_2 = -1$$
$$\bar{x} + 1 = -x$$

- Example: negate +2
  - $+2 = 0000\ 0000 \dots 0010_2$
  - $-2 = 1111\ 1111 \dots 1101_2 + 1$   
 $= 1111\ 1111 \dots 1110_2$

# SIGN EXTENSION

- Representing a number using more bits
  - Preserve the numeric value
- Replicate the sign bit to the left
  - c.f. unsigned values: extend with 0s
- Examples: 8-bit to 16-bit
  - +2: 0000 0010 => 0000 0000 0000 0010
  - -2: 1111 1110 => 1111 1111 1111 1110
- In RISC-V instruction set
  - lb: sign-extend loaded byte
  - lbu: zero-extend loaded byte

# REPRESENTING INSTRUCTIONS

- Instructions are encoded in binary
  - Called machine code
- RISC-V instructions
  - Encoded as 32-bit instruction words
  - A small number of formats encoding operation code (opcode), register numbers, ...
  - Regularity!

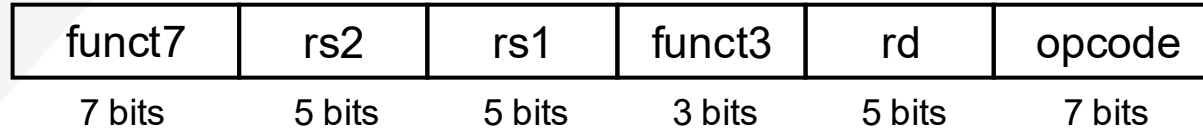
# HEXADECIMAL

- Base 16
  - Compact representation of bit strings
  - 4 bits per hex digit

|   |      |   |      |   |      |   |      |
|---|------|---|------|---|------|---|------|
| 0 | 0000 | 4 | 0100 | 8 | 1000 | c | 1100 |
| 1 | 0001 | 5 | 0101 | 9 | 1001 | d | 1101 |
| 2 | 0010 | 6 | 0110 | a | 1010 | e | 1110 |
| 3 | 0011 | 7 | 0111 | b | 1011 | f | 1111 |

- Example: eca8 6420
  - 1110 1100 1010 1000 0110 0100 0010 0000

# RISC-V R-FORMAT INSTRUCTIONS



- Instruction fields
  - opcode: operation code
  - rd: destination register number
  - funct3: 3-bit function code (additional opcode)
  - rs1: the first source register number
  - rs2: the second source register number
  - funct7: 7-bit function code (additional opcode)

## R-FORMAT EXAMPLE

|        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|
| funct7 | rs2    | rs1    | funct3 | rd     | opcode |
| 7 bits | 5 bits | 5 bits | 3 bits | 5 bits | 7 bits |

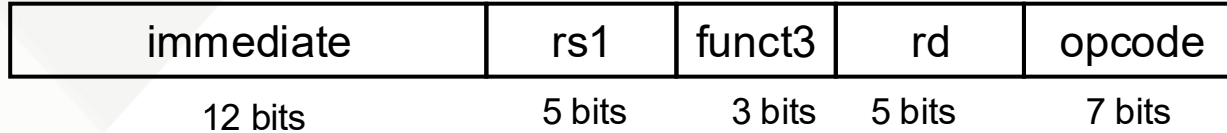
add x9, x20, x21

|         |       |       |     |       |         |
|---------|-------|-------|-----|-------|---------|
| 0       | 21    | 20    | 0   | 9     | 51      |
| 0000000 | 10101 | 10100 | 000 | 01001 | 0110011 |

0000 0001 0101 1010 0000 0100 1011 0011<sub>2</sub> = 015A04B3<sub>16</sub>

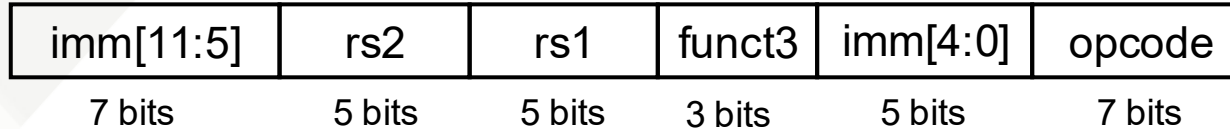
Binary representation is not easy to understand!

# RISC-V I-FORMAT INSTRUCTIONS



- Immediate arithmetic and load instructions
  - rs1: source or base address register number
  - immediate: constant operand, or offset added to base address
    - 2s-complement, sign extended
- *Design Principle 3: Good design demands good compromises*
  - Different formats complicate decoding, but allow 32-bit instructions uniformly
  - Keep formats as similar as possible

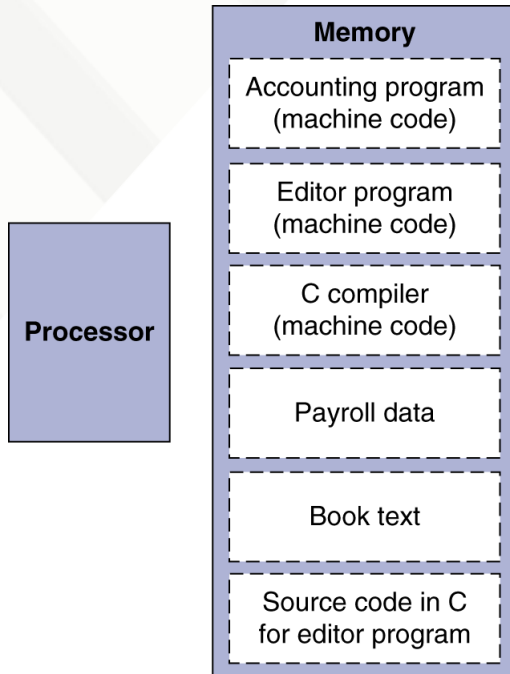
# RISC-V S-FORMAT INSTRUCTIONS



- Different immediate format for store instructions
  - rs1: base address register number
  - rs2: source operand register number
  - immediate: offset added to base address
    - Split so that rs1 and rs2 fields always in the same place

# STORED PROGRAM COMPUTERS

## The BIG Picture



- Instructions represented in binary, just like data
- Instructions and data stored in memory
- Programs can operate on programs
  - e.g., compilers, linkers, ...
- Binary compatibility allows compiled programs to work on different computers
  - Standardized ISAs

# LOGICAL OPERATIONS

- Instructions for bitwise manipulation

| Operation      | C  | Java | RISC-V    |
|----------------|----|------|-----------|
| Shift left     | << | <<   | slli      |
| Shift right    | >> | >>>  | srlr      |
| Bit-by-bit AND | &  | &    | and, andi |
| Bit-by-bit OR  |    |      | or, ori   |
| Bit-by-bit XOR | ^  | ^    | xor, xori |
| Bit-by-bit NOT | ~  | ~    |           |

- Useful for extracting and inserting groups of bits in a word

# SHIFT OPERATIONS

|        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|
| funct6 | immed  | rs1    | funct3 | rd     | opcode |
| 6 bits | 6 bits | 5 bits | 3 bits | 5 bits | 6 bits |

- immed: how many positions to shift
- Shift left logical
  - Shift left and fill with 0 bits
  - $sll\ i\ by\ i\ bits$  multiplies by  $2^i$
- Shift right logical
  - Shift right and fill with 0 bits
  - $srl\ i\ by\ i\ bits$  divides by  $2^i$  (unsigned only)

# AND OPERATIONS

- Useful to mask bits in a word
  - Select some bits, clear others to 0

and x9, x10, x11

|     |                                     |
|-----|-------------------------------------|
| x10 | 00000000 00000000 00001101 11000000 |
| x11 | 00000000 00000000 00111100 00000000 |
| x9  | 00000000 00000000 00001100 00000000 |

# OR OPERATIONS

- Useful to include bits in a word
  - Set some bits to 1, leave others unchanged

or x9, x10, x11

|     |          |          |          |          |
|-----|----------|----------|----------|----------|
| x10 | 00000000 | 00000000 | 00001101 | 11000000 |
| x11 | 00000000 | 00000000 | 00111100 | 00000000 |
| x9  | 00000000 | 00000000 | 00111101 | 11000000 |

# XOR OPERATIONS

- Differencing operation
  - Set some bits to 1, leave others unchanged
- `xor x9, x10, x12 // NOT operation`

|     |          |          |          |          |
|-----|----------|----------|----------|----------|
| x10 | 00000000 | 00000000 | 00001101 | 11000000 |
| x12 | 11111111 | 11111111 | 11111111 | 11111111 |
| x9  | 11111111 | 11111111 | 11110010 | 00111111 |

# CONDITIONAL OPERATIONS

- Branch to a labeled instruction if a condition is true
  - Otherwise, continue sequentially
- `beq rs1, rs2, L1`
  - if ( $rs1 == rs2$ ) branch to instruction labeled L1
- `bne rs1, rs2, L1`
  - if ( $rs1 \neq rs2$ ) branch to instruction labeled L1

# COMPILING IF STATEMENTS

- C code:

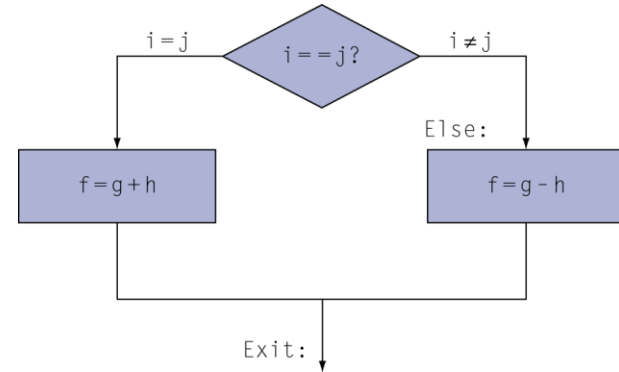
```
if (i==j) f = g+h;  
else f = g-h;
```

— f, g, ... in x19, x20, ...

- Compiled RISC-V code:

```
    bne x22, x23, Else  
    add x19, x20, x21  
    beq x0, x0, Exit // unconditional  
Else: sub x19, x20, x21  
Exit: ...
```

Assembler calculates addresses



## COMPILING LOOP STATEMENTS

- C code:

```
while (save[i] == k) i += 1;
```

— i in x22, k in x24, address of save in x25

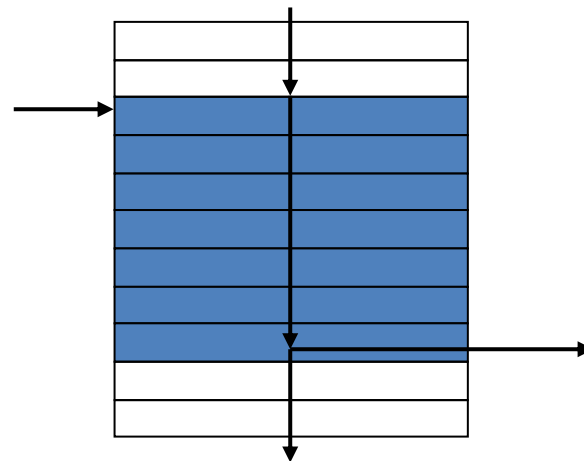
- Compiled RISC-V code:

```
Loop: slli x10, x22, 2  
      add x10, x10, x25  
      lw x9, 0(x10)  
      bne x9, x24, Exit  
      addi x22, x22, 1  
      beq x0, x0, Loop
```

```
Exit: ...
```

# BASIC BLOCKS

- A basic block is a sequence of instructions with
  - No embedded branches (except at end)
  - No branch targets (except at beginning)
- A compiler identifies basic blocks for optimization
- An advanced processor can accelerate execution of basic blocks



## MORE CONDITIONAL OPERATIONS

- `blt rs1, rs2, L1`
  - if ( $rs1 < rs2$ ) branch to instruction labeled L1
- `bge rs1, rs2, L1`
  - if ( $rs1 \geq rs2$ ) branch to instruction labeled L1
- Example
  - if ( $a > b$ ) `a += 1;`
  - `a in x22, b in x23`  
`bge x23, x22, Exit // branch if b  $\geq$  a`  
`addi x22, x22, 1`

Exit:

## SIGNED VS. UNSIGNED

- Signed comparison: `blt`, `bge`
- Unsigned comparison: `bltu`, `bgeu`
- Example

– `x22 = 1111 1111 1111 1111 1111 1111 1111 1111`

– `x23 = 0000 0000 0000 0000 0000 0000 0000 0001`

– `x22 < x23 // signed`

- `-1 < +1`

– `x22 > x23 // unsigned`

- `+4,294,967,295 > +1`

# PROCEDURE CALLING

- Steps required
  - Place parameters in registers x10 to x17
  - Transfer control to procedure
  - Acquire storage for procedure
  - Perform procedure's operations
  - Place result in register for caller
  - Return to place of call (address in x1)

| Register | ABI Name | Description                      | Saver  |
|----------|----------|----------------------------------|--------|
| x0       | zero     | Hard-wired zero                  | —      |
| x1       | ra       | Return address                   | Caller |
| x2       | sp       | Stack pointer                    | Callee |
| x3       | gp       | Global pointer                   | —      |
| x4       | tp       | Thread pointer                   | —      |
| x5–7     | t0–2     | Temporaries                      | Caller |
| x8       | s0/fp    | Saved register/frame pointer     | Callee |
| x9       | s1       | Saved register                   | Callee |
| x10–11   | a0–1     | Function arguments/return values | Caller |
| x12–17   | a2–7     | Function arguments               | Caller |
| x18–27   | s2–11    | Saved registers                  | Callee |
| x28–31   | t3–6     | Temporaries                      | Caller |
| f0–7     | ft0–7    | FP temporaries                   | Caller |
| f8–9     | fs0–1    | FP saved registers               | Callee |
| f10–11   | fa0–1    | FP arguments/return values       | Caller |
| f12–17   | fa2–7    | FP arguments                     | Caller |
| f18–27   | fs2–11   | FP saved registers               | Callee |
| f28–31   | ft8–11   | FP temporaries                   | Caller |

Table 18.2: RISC-V calling convention register usage.

# PROCEDURE CALL INSTRUCTIONS

- Procedure call: jump and link

**jal x1, ProcedureLabel**

- Address of following instruction put in x1
- Jumps to the target address

- Procedure return: jump and link register

**jalr x0, 0(x1)**

- Unlike jal, but jumps to 0 + address in x1
- Use x0 as rd (x0 cannot be changed)
- Can also be used for computed jumps
  - e.g., for case/switch statements

# LEAF PROCEDURE EXAMPLE

- C code:

```
int leaf_example (int g, int h, int i, int j) {  
    int f;  
    f = (g + h) - (i + j);  
    return f;  
}
```

- Arguments g, ..., j in x10, ..., x13
- f in x20
- Temporaries x5, x6
- Need to save x5, x6, x20 on stack

## LEAF PROCEDURE EXAMPLE

- RISC-V code:

leaf\_example:

```
addi sp, sp, -12
sw    x5, 8(sp)
sw    x6, 4(sp)
sw    x20, 0(sp)
add   x5, x10, x11
add   x6, x12, x13
sub   x20, x5, x6
addi  x10, x20, 0
lw    x20, 0(sp)
lw    x6, 4(sp)
lw    x5, 8(sp)
addi  sp, sp, 12
jalr  x0, 0(x1)
```

Save x5, x6, x20 on stack

$x5 = g + h$

$x6 = i + j$

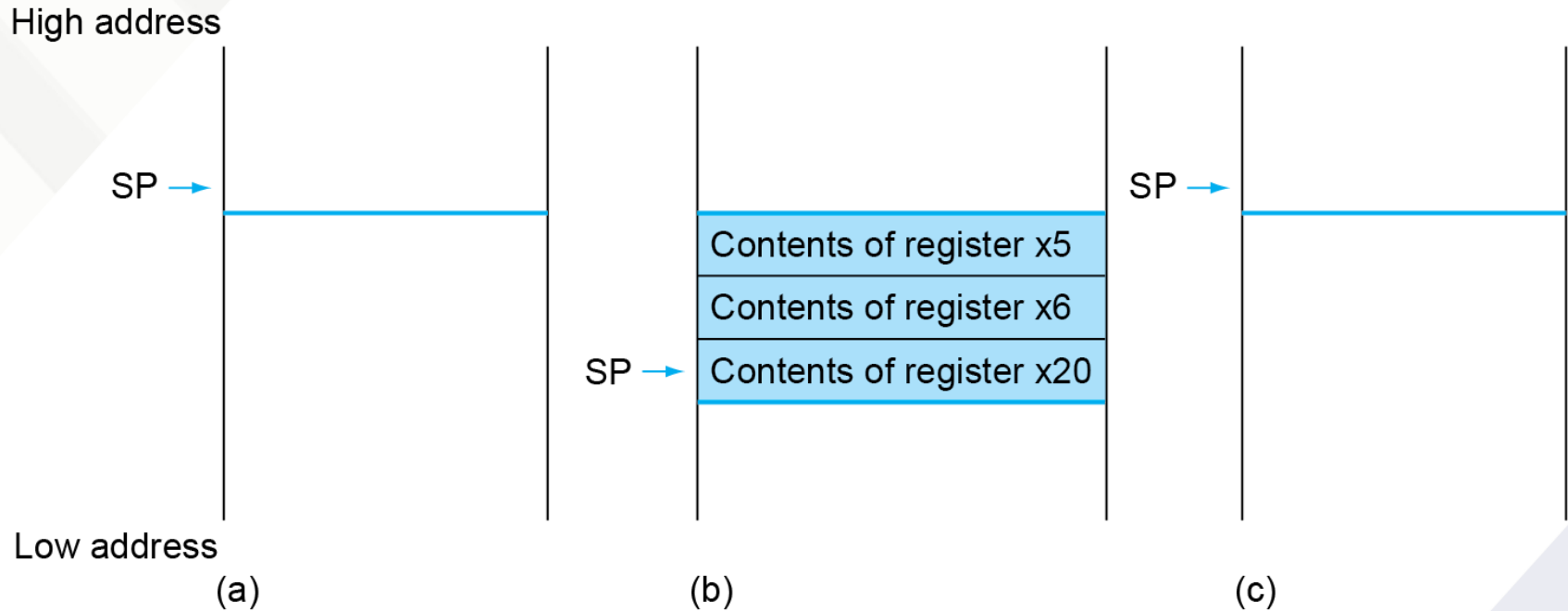
$f = x5 - x6$

Copy f to return register

Restore x5, x6, x20 from stack

Return to caller

# STACK VALUES DURING PROCEDURE CALL



## REGISTER USAGE

- x5 – x7, x28 – x31: temporary registers
  - Not preserved by the callee
- x8 – x9, x18 – x27: saved registers
  - If used, the callee saved and restores them

# NON-LEAF PROCEDURES

- Procedures that call other procedures
- For nested call, caller needs to save on the stack:
  - Its return address
  - Any arguments and temporaries needed after the call
- Restore from the stack after the call

## NON-LEAF PROCEDURE EXAMPLE

- C code:

```
int fact (int n)
{
    if (n < 1) return f;
    else return n * fact(n - 1);
}
```

- Argument n in x10
- Result in x10

## NON-LEAF PROCEDURE EXAMPLE

- RISC-V code:

fact:

```
    addi sp, sp, -8
    sw    x1, 4(sp)
    sw    x10, 0(sp)
    addi x5, x10, -1
    bge x5, x0, L1
    addi x10, x0, 1
    addi sp, sp, 16
    jalr x0, 0(x1)
L1: addi x10, x10, -1
    jal  x1, fact
    lw   x10, 0(sp)
    lw   x1, 4(sp)
    addi sp, sp, 8
    mul  x10, x10, x6
    jalr x0, 0(x1)
```

```
save return address and n on stack
save return address
save argument
x5 = n - 1
if n >= 1, go to L1
else, set return value to 1
pop stack, don't bother restoring values
return
n = n - 1
call fact(n-1)
restore caller's n
restore caller's return address
pop 2 items from stack
return n * fact(n-1)
return
```

# MEMORY LAYOUT

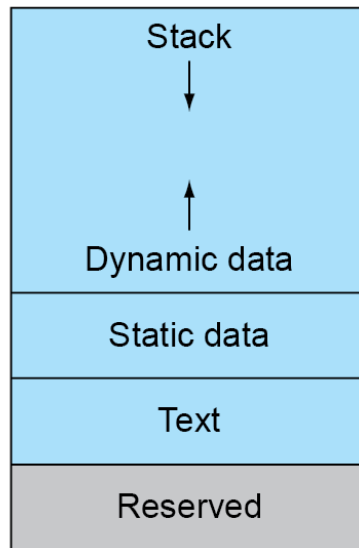
- Text: program code
- Static data: global variables
  - e.g., static variables in C, constant arrays and strings
  - x3 (global pointer) initialized to address allowing  $\pm$ offsets into this segment
- Dynamic data: heap
  - e.g., malloc in C, new in Java
- Stack: automatic storage

SP → 0000 003f ffff fff0<sub>hex</sub>

0000 0000 1000 0000<sub>hex</sub>

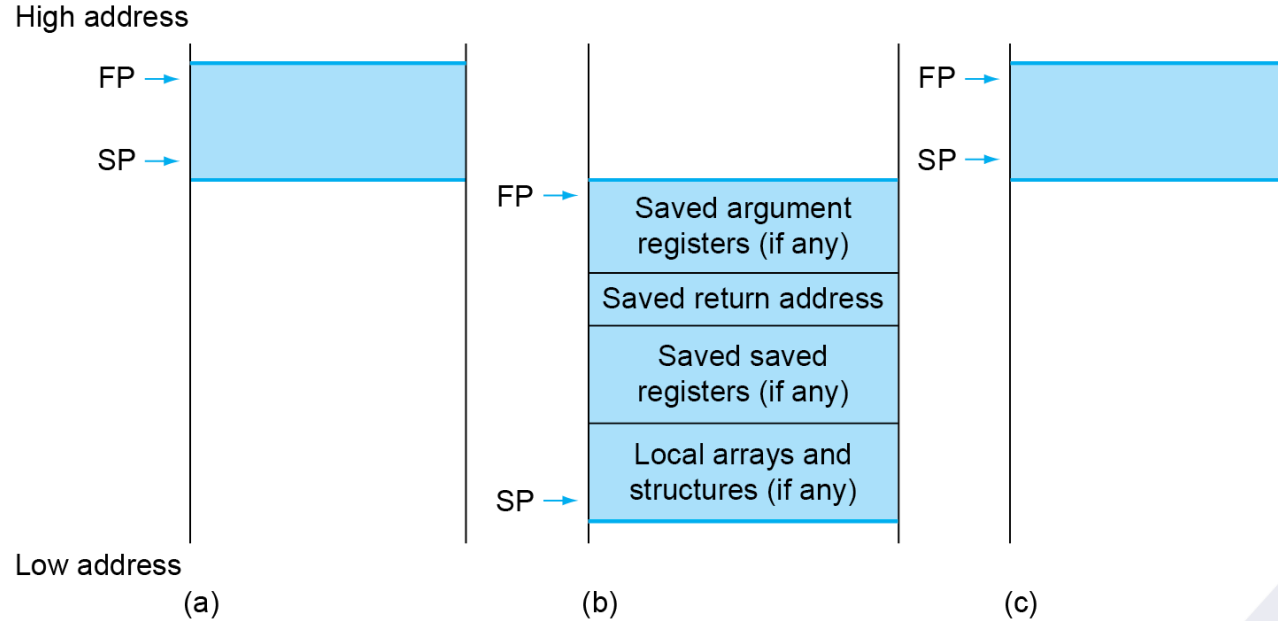
PC → 0000 0000 0040 0000<sub>hex</sub>

0



# LOCAL DATA ON THE STACK

- Local data allocated by callee
  - e.g., C automatic variables, which are declared inside a block
- Procedure frame (activation record)
  - Used by some compilers to manage stack storage



# BYTE/HALFWORD/WORD OPERATIONS

- RISC-V byte/halfword/word load/store
  - Load byte/halfword/word: Sign extend to 64 bits in rd
    - `lb rd, offset(rs1)`
    - `lh rd, offset(rs1)`
    - `lw rd, offset(rs1)`
  - Load byte/halfword/word unsigned: zero extend to 64 bits in rd
    - `lbu rd, offset(rs1)`
    - `lhu rd, offset(rs1)`
    - `lwu rd, offset(rs1)`
  - Store byte/halfword/word: store rightmost 8/16/32 bits
    - `sb rs2, offset(rs1)`
    - `sh rs2, offset(rs1)`
    - `sw rs2, offset(rs1)`

## STRING COPY EXAMPLE

- C code (naïve):
  - Null-terminated string

```
void strcpy (char x[], char y[])
{
    size_t i;
    i = 0;
    while ((x[i]=y[i])!='\0')
        i += 1;
}
```

## STRING COPY EXAMPLE

- RISC-V code:

```
strcpy:
    addi sp, sp, -4           // adjust stack for 1 word
    sw   x19, 0(sp)          // push x19
    add  x19, x0, x0          // i = 0
L1:    add x5, x19, x10        // x5 = addr of y[i]
        lbu x6, 0(x5)         // x6 = y[i]
        add x7, x19, x10      // x7 = addr of x[i]
        sb  x6, 0(x7)         // x[i] = y[i]
        beq x6, x0, L2        // if y[i] == 0 then exit
        addi x19, x19, 1      // i = i + 1
        jal x0, L1            // next iteration of loop
L2:    lw   x19, 0(sp)         // restore saved x19
        addi sp, sp, 4        // pop 1 item (word) from stack
        jalr x0, 0(x1)        // and return
```

## 32-BIT CONSTANTS

- Most constants are small
  - 12-bit immediate is sufficient
- For the occasional 32-bit constant

`lui rd, constant`

- Copies 20-bit constant to bits [31:12] of rd
- Extends bit 31 to bits [63:32]
- Clears bits [11:0] of rd to 0

`lui x19, 976 // 0x003D0`

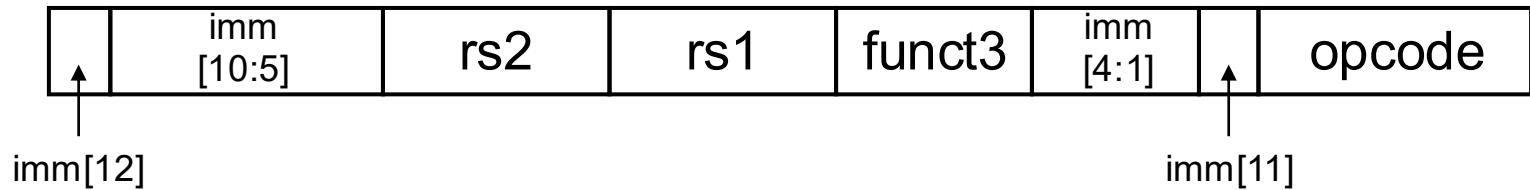
|                          |                |
|--------------------------|----------------|
| 0000 0000 0011 1101 0000 | 0000 0000 0000 |
|--------------------------|----------------|

`addi x19,x19,1280 // 0x500`

|                          |                |
|--------------------------|----------------|
| 0000 0000 0011 1101 0000 | 0101 0000 0000 |
|--------------------------|----------------|

# BRANCH ADDRESSING

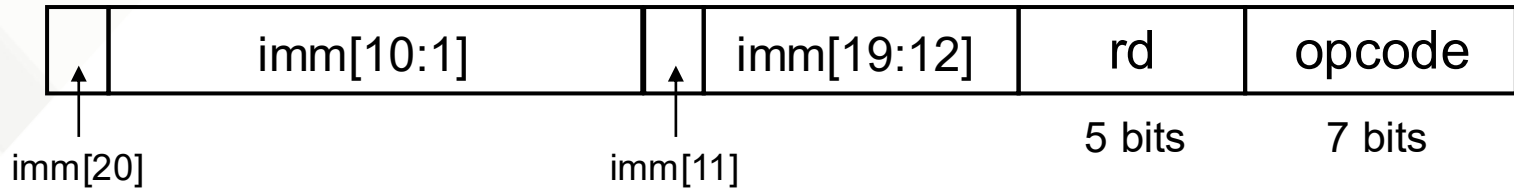
- Branch instructions specify
  - Opcode, two registers, target address
- Most branch targets are near branch
  - Forward or backward
- SB format:



- PC-relative addressing
  - Target address =  $PC + \text{immediate} * 2$

# JUMP ADDRESSING

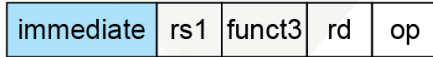
- Jump and link (`jal`) targets uses 20-bit immediate for larger range
- UJ format:



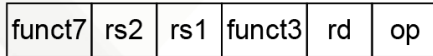
- For long jumps, e.g., to 32-bit absolute address
  - `lui`: load address[31:12] to temp register
  - `jalr`: add address[11:0] and jump to target

# RISC-V ADDRESSING SUMMARY

## 1. Immediate addressing



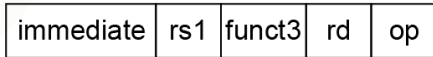
## 2. Register addressing



Registers

Register

## 3. Base addressing



Memory

Register

+

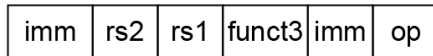
Byte

Halfword

Word

Doubleword

## 4. PC-relative addressing



Memory

PC

+

Word

# RISC-V ENCODING SUMMARY

| Name<br>(Field Size) | Field                       |        |        |        |               |        | Comments                      |
|----------------------|-----------------------------|--------|--------|--------|---------------|--------|-------------------------------|
|                      | 7 bits                      | 5 bits | 5 bits | 3 bits | 5 bits        | 7 bits |                               |
| R-type               | funct7                      | rs2    | rs1    | funct3 | rd            | opcode | Arithmetic instruction format |
| I-type               | immediate[11:0]             |        | rs1    | funct3 | rd            | opcode | Loads & immediate arithmetic  |
| S-type               | immed[11:5]                 | rs2    | rs1    | funct3 | immed[4:0]    | opcode | Stores                        |
| SB-type              | immed[12,10:5]              | rs2    | rs1    | funct3 | immed[4:1,11] | opcode | Conditional branch format     |
| UJ-type              | immediate[20,10:1,11,19:12] |        |        |        | rd            | opcode | Unconditional jump format     |
| U-type               | immediate[31:12]            |        |        |        | rd            | opcode | Upper immediate format        |

# SYNCHRONIZATION

- Two processors sharing an area of memory
  - P1 writes, then P2 reads
  - Data race if P1 and P2 don't synchronize
    - Result depends on order of accesses
    - E.g., P1's local sum = 3, P2's local sum = 4, global sum=?
- Hardware support required
  - Atomic read/write memory operation
  - No other access to the location allowed between the read and write
- Could be a single instruction
  - e.g., atomic swap of register  $\Leftrightarrow$  memory
  - Or an atomic pair of instructions

## SYNCHRONIZATION IN RISC-V

- Load reserved: `lr.w rd, offset(rs1)`
  - Load from address in rs1 to rd
  - Place reservation on memory address
- Store conditional: `sc.w rd,(rs1),rs2`
  - Store from rs2 to address in rs1
  - Succeeds if location not changed since the lr.d
    - Returns 0 in rd
  - Fails if location is changed
    - Returns non-zero value in rd

## SYNCHRONIZATION IN RISC-V

- Example 1: atomic swap (to test/set lock variable)

```
again:   lr.w x10, (x20)
         sc.w x11, (x20), x23 // x11 = status
         bne  x11, x0, again  // branch if store failed
         addi x23, x10, 0     // x23 = loaded value
```

- Example 2: lock

```
again:   addi x12, x0, 1      // copy locked value
         lr.w x10, (x20)      // read lock
         bne  x10, x0, again  // check if it is 0 yet
         sc.w x11, (x20), x12 // attempt to store
         bne  x11, x0, again  // branch if fails
```

- Unlock:

```
sw      x0, 0(x20)           // free lock
```

## C SORT EXAMPLE

- Illustrates use of assembly instructions for a C bubble sort function
- Swap procedure (leaf)

```
void swap(int v[], int k)
{
    int temp;
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;
}
```

— v in x10, k in x11, temp in x5

## THE PROCEDURE SWAP

swap:

```
slli x6,x11,2    // reg x6 = k * 4
add  x6,x10,x6    // reg x6 = v + (k * 4)
lw   x5,0(x6)     // reg x5 (temp) = v[k]
lw   x7,4(x6)     // reg x7 = v[k + 1]
sw   x7,0(x6)     // v[k] = reg x7
sw   x5,4(x6)     // v[k+1] = reg x5 (temp)
jalr x0,0(x1)     // return to calling routine
```

## THE SORT PROCEDURE IN C

- Non-leaf (calls swap)

```
void sort (int v[], size_t n)
{
    size_t i, j;
    for (i = 0; i < n; i += 1) {
        for (j = i - 1;
             j >= 0 && v[j] > v[j + 1];
             j -= 1) {
            swap(v, j);
        }
    }
}
```

— v in x10, n in x11, i in x19, j in x20

## THE OUTER LOOP

- Skeleton of outer loop:

- for ( $i = 0; i < n; i += 1$ ) {

- li x19,0 // same as addi x19, x0, 0;  $i = 0$

- for1tst:

- bge x19,x11,exit1 // go to exit1 if  $x19 \geq x11$  ( $i \geq n$ )

- (body of outer for-loop)

- addi x19,x19,1 //  $i += 1$

- j for1tst // branch to test of outer loop

- exit1:

# THE INNER LOOP

- Skeleton of inner loop:

- for ( $j = i - 1$ ;  $j \geq 0$  &&  $v[j] > v[j + 1]$ ;  $j - = 1$ ) {

```
    addi x20,x19,-1    // j = i -1
for2tst:
    blt  x20,x0,exit2  // go to exit2 if x20 < 0 (j < 0)
    slli x5,x20,2      // reg x5 = j * 4
    add  x5,x10,x5      // reg x5 = v + (j * 4)
    lw   x6,0(x5)       // reg x6 = v[j]
    lw   x7,4(x5)       // reg x7 = v[j + 1]
    ble  x6,x7,exit2    // go to exit2 if x6 ≤ x7
    mv   x21, x10       // copy parameter x10 into x21
    mv   x22, x11       // copy parameter x11 into x22
    mv   x10, x21       // first swap parameter is v
    mv   x11, x20       // second swap parameter is j
    jal  x1,swap        // call swap
    addi x20,x20,-1     // j -= 1
    j    for2tst        // branch to test of inner loop
exit2:
```

# PRESERVING REGISTERS

- Preserve saved registers:

```
addi sp,sp,-20 // make room on stack for 5 regs (5*4)
sw   x1,16(sp) // save x1 on stack
sw   x22,12(sp) // save x22 on stack
sw   x21,8(sp)  // save x21 on stack
sw   x20,4(sp)  // save x20 on stack
sw   x19,0(sp)  // save x19 on stack
```

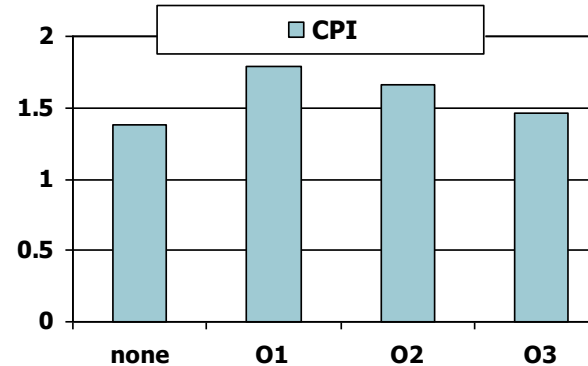
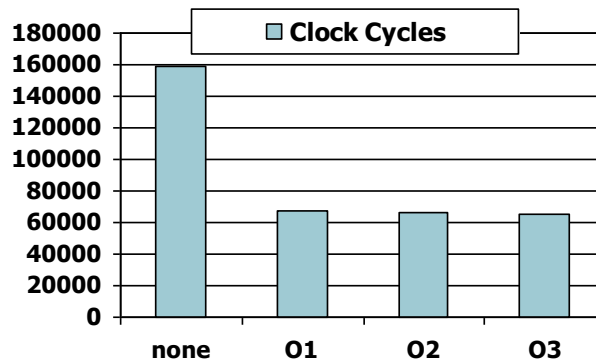
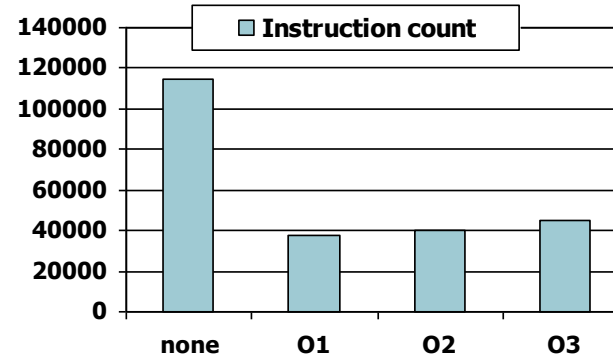
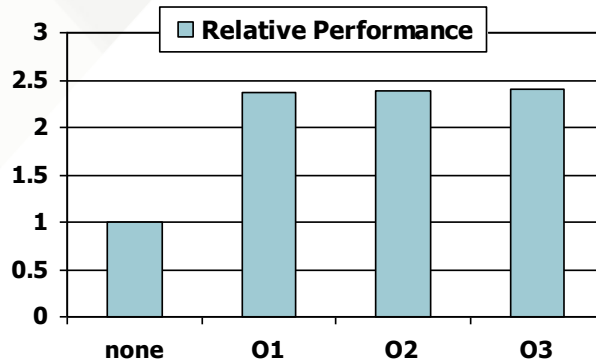
- Restore saved registers:

exit1:

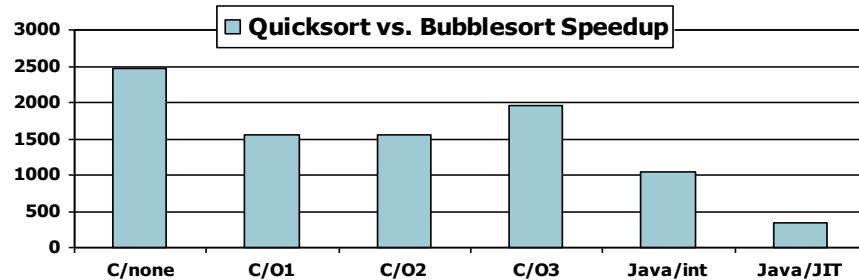
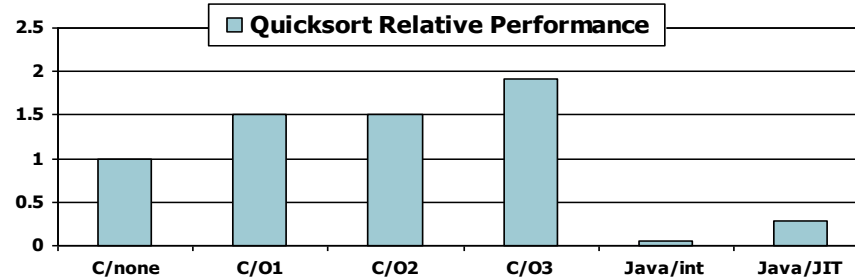
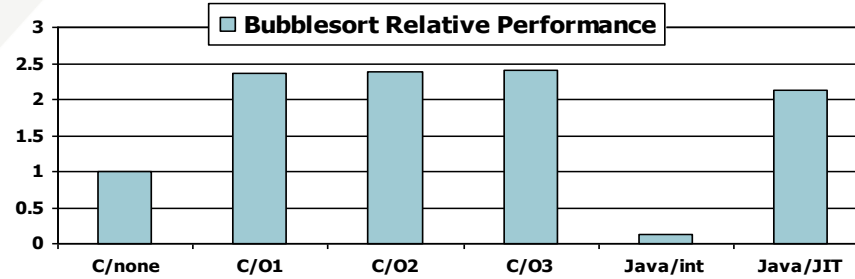
```
lw   x19,0(sp) // restore x19 from stack
lw   x20,4(sp) // restore x20 from stack
lw   x21,8(sp) // restore x21 from stack
lw   x22,12(sp) // restore x22 from stack
lw   x1,16(sp) // restore x1 from stack
addi sp,sp,20  // restore stack pointer
jalr x0,0(x1)
```

# EFFECT OF COMPILER OPTIMIZATION

Compiled with gcc for Pentium 4 under Linux



# EFFECT OF LANGUAGE AND ALGORITHM



# ARRAYS VS. POINTERS

- Array indexing involves
  - Multiplying index by element size
  - Adding to array base address
- Pointers correspond directly to memory addresses
  - Can avoid indexing complexity

## EXAMPLE: CLEARING AND ARRAY

```
clear1(int array[], int size) {
    int i;
    for (i = 0; i < size; i += 1)
        array[i] = 0;
}
```

```
li    x5,0          // i = 0
loop1:
slli  x6,x5,2        // x6 = i * 4
add   x7,x10,x6      // x7 = address
                        // of array[i]
sw    x0,0(x7)       // array[i] = 0
addi  x5,x5,1        // i = i + 1
blt   x5,x11,loop1   // if (i<size)
                        // go to loop1
```

```
clear2(int *array, int size) {
    int *p;
    for (p = &array[0]; p < &array[size];
        p = p + 1)
        *p = 0;
}
```

```
mv    x5,x10         // p = address
                        // of array[0]
slli  x6,x11,2        // x6 = size * 4
add   x7,x10,x6      // x7 = address
                        // of array[size]
loop2:
sw    x0,0(x5)       // Memory[p] = 0
addi  x5,x5,8        // p = p + 8
bltu  x5,x7,loop2    // if (p<&array[size])
                        // go to loop2
```

## COMPARISON OF ARRAY VS. PTR

- Multiply “strength reduced” to shift
- Array version requires shift to be inside loop
  - Part of index calculation for incremented  $i$
  - c.f. incrementing pointer
- Compiler can achieve same effect as manual use of pointers
  - Induction variable elimination, e.g.,  $i$  variable
  - Better to make program clearer and safer

# MIPS INSTRUCTIONS

- MIPS: commercial predecessor to RISC-V
- Similar basic set of instructions
  - 32-bit instructions
  - 32 general purpose registers, register 0 is always 0
  - 32 floating-point registers
  - Memory accessed only by load/store instructions
    - Consistent use of addressing modes for all data sizes
- Different conditional branches
  - For <, <=, >, >=
  - RISC-V: blt, bge, bltu, bgeu
  - MIPS: slt, sltu (set less than, result is 0 or 1)
    - Then use beq, bne to complete the branch

# INSTRUCTION ENCODING

## Register-register

|        |           |    |    |    |        |    |        |    |           |   |          |   |           |  |
|--------|-----------|----|----|----|--------|----|--------|----|-----------|---|----------|---|-----------|--|
|        | 31        | 25 | 24 | 20 | 19     | 15 | 14     | 12 | 11        | 7 | 6        | 0 |           |  |
| RISC-V | funct7(7) |    |    |    | rs2(5) |    | rs1(5) |    | funct3(3) |   | rd(5)    |   | opcode(7) |  |
|        | 31        | 26 | 25 | 21 | 20     | 16 | 15     | 11 | 10        | 6 | 5        | 0 |           |  |
| MIPS   | Op(6)     |    |    |    | Rs1(5) |    | Rs2(5) |    | Rd(5)     |   | Const(5) |   | Opx(6)    |  |

## Load

|        |               |    |    |    |        |    |           |   |           |   |           |   |
|--------|---------------|----|----|----|--------|----|-----------|---|-----------|---|-----------|---|
|        | 31            | 20 | 19 | 15 | 14     | 12 | 11        | 7 | 6         | 0 |           |   |
| RISC-V | immediate(12) |    |    |    | rs1(5) |    | funct3(3) |   | rd(5)     |   | opcode(7) |   |
|        | 31            | 26 | 25 | 21 | 20     | 16 | 15        |   |           |   |           | 0 |
| MIPS   | Op(6)         |    |    |    | Rs1(5) |    | Rs2(5)    |   | Const(16) |   |           |   |

## Store

|        |              |    |    |    |        |    |        |    |           |   |              |   |           |   |
|--------|--------------|----|----|----|--------|----|--------|----|-----------|---|--------------|---|-----------|---|
|        | 31           | 25 | 24 | 20 | 19     | 15 | 14     | 12 | 11        | 7 | 6            | 0 |           |   |
| RISC-V | immediate(7) |    |    |    | rs2(5) |    | rs1(5) |    | funct3(3) |   | immediate(5) |   | opcode(7) |   |
|        | 31           | 26 | 25 | 21 | 20     | 16 | 15     |    |           |   |              |   |           | 0 |
| MIPS   | Op(6)        |    |    |    | Rs1(5) |    | Rs2(5) |    | Const(16) |   |              |   |           |   |

## Branch

|        |              |    |    |    |        |    |            |    |           |   |              |   |           |   |
|--------|--------------|----|----|----|--------|----|------------|----|-----------|---|--------------|---|-----------|---|
|        | 31           | 25 | 24 | 20 | 19     | 15 | 14         | 12 | 11        | 7 | 6            | 0 |           |   |
| RISC-V | immediate(7) |    |    |    | rs2(5) |    | rs1(5)     |    | funct3(3) |   | immediate(5) |   | opcode(7) |   |
|        | 31           | 26 | 25 | 21 | 20     | 16 | 15         |    |           |   |              |   |           | 0 |
| MIPS   | Op(6)        |    |    |    | Rs1(5) |    | Opx/Rs2(5) |    | Const(16) |   |              |   |           |   |

# OTHER RISC-V INSTRUCTIONS

- Base integer instructions (RV64I)
  - Those previously described, plus
  - `auipc rd, imm` //  $rd = (imm \ll 12) + pc$ 
    - follow by `jalr` (adds 12-bit `imm`) for long jump
  - `slt`, `sltu`, `slti`, `sltiu`: set less than (like MIPS)
  - `addw`, `subw`, `addiw`: 32-bit add/sub
  - `sllw`, `srlw`, `sllw`, `slliw`, `srliw`, `sraiw`: 32-bit shift
- 32-bit variant: RV32I
  - registers are 32-bits wide, 32-bit operations

**Additional Instructions in RISC-V Base Architecture**

| Instruction                          | Name               | Format | Description                                             |
|--------------------------------------|--------------------|--------|---------------------------------------------------------|
| Add upper immediate to PC            | <code>auipc</code> | U      | Add 20-bit upper immediate to PC; write sum to register |
| Set if less than                     | <code>slt</code>   | R      | Compare registers; write Boolean result to register     |
| Set if less than, unsigned           | <code>sltu</code>  | R      | Compare registers; write Boolean result to register     |
| Set if less than, immediate          | <code>slti</code>  | I      | Compare registers; write Boolean result to register     |
| Set if less than immediate, unsigned | <code>sltiu</code> | I      | Compare registers; write Boolean result to register     |

# INSTRUCTION SET EXTENSIONS

- M: integer multiply, divide, remainder
- A: atomic memory operations
- F: single-precision floating point
- D: double-precision floating point
- C: compressed instructions
  - 16-bit encoding for frequently used instructions

**RISC-V Base and Extensions**

| Mnemonic | Description                     | Insn. Count |
|----------|---------------------------------|-------------|
| I        | Base architecture               | 51          |
| M        | Integer multiply/divide         | 13          |
| A        | Atomic operations               | 22          |
| F        | Single-precision floating point | 30          |
| D        | Double-precision floating point | 32          |
| C        | Compressed instructions         | 36          |

# RISC-V INSTRUCTION SET COVERED SO FAR

| RISC-V Instructions               | Name | Format |
|-----------------------------------|------|--------|
| Add                               | add  | R      |
| Subtract                          | sub  | R      |
| Add immediate                     | addi | I      |
| Load word                         | lw   | I      |
| Load word, unsigned               | lwu  | I      |
| Store word                        | sw   | S      |
| Load halfword                     | lh   | I      |
| Load halfword, unsigned           | lhu  | I      |
| Store halfword                    | sh   | S      |
| Load byte                         | lb   | I      |
| Load byte, unsigned               | lbu  | I      |
| Store byte                        | sb   | S      |
| Load reserved                     | lr.d | R      |
| Store conditional                 | sc.d | R      |
| Load upper immediate              | lui  | U      |
| And                               | and  | R      |
| Inclusive or                      | or   | R      |
| Exclusive or                      | xor  | R      |
| And immediate                     | andi | I      |
| Inclusive or immediate            | ori  | I      |
| Exclusive or immediate            | xori | I      |
| Shift left logical                | sll  | R      |
| Shift right logical               | srl  | R      |
| Shift right arithmetic            | sra  | R      |
| Shift left logical immediate      | slli | I      |
| Shift right logical immediate     | slli | I      |
| Shift right arithmetic immediate  | srai | I      |
| Branch if equal                   | beq  | SB     |
| Branch if not equal               | bne  | SB     |
| Branch if less than               | blt  | SB     |
| Branch if greater or equal        | bge  | SB     |
| Branch if less, unsigned          | bltu | SB     |
| Branch if greater/equal, unsigned | bgeu | SB     |
| Jump and link                     | jal  | UJ     |
| Jump and link register            | jalr | I      |

## RISC-V operands

| Name                         | Example                                          | Comments                                                                                                                                                                       |
|------------------------------|--------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 32 registers                 | x0-x31                                           | Fast locations for data. In RISC-V, data must be in registers to perform arithmetic. Register x0 always equals 0.                                                              |
| 2 <sup>30</sup> memory words | Memory[0], Memory[4], ..., Memory[4,294,967,292] | Accessed only by data transfer instructions. RISC-V uses byte addresses, so sequential word accesses differ by 4. Memory holds data structures, arrays, and spilled registers. |

## RISC-V assembly language

| Category      | Instruction                      | Example           | Meaning                   | Comments                                   |
|---------------|----------------------------------|-------------------|---------------------------|--------------------------------------------|
| Arithmetic    | Add                              | add x5, x6, x7    | x5 = x6 + x7              | Three register operands; add               |
|               | Subtract                         | sub x5, x6, x7    | x5 = x6 - x7              | Three register operands; subtract          |
|               | Add immediate                    | addi x5, x6, 20   | x5 = x6 + 20              | Used to add constants                      |
| Data transfer | Load word                        | lw x5, 40(x6)     | x5 = Memory[x6 + 40]      | Word from memory to register               |
|               | Load word, unsigned              | lwu x5, 40(x6)    | x5 = Memory[x6 + 40]      | Unsigned word from memory to register      |
|               | Store word                       | sw x5, 40(x6)     | Memory[x6 + 40] = x5      | Word from register to memory               |
|               | Load halfword                    | lh x5, 40(x6)     | x5 = Memory[x6 + 40]      | Halfword from memory to register           |
|               | Load halfword, unsigned          | lhu x5, 40(x6)    | x5 = Memory[x6 + 40]      | Unsigned halfword from memory to register  |
|               | Store halfword                   | sh x5, 40(x6)     | Memory[x6 + 40] = x5      | Halfword from register to memory           |
|               | Load byte                        | lb x5, 40(x6)     | x5 = Memory[x6 + 40]      | Byte from memory to register               |
|               | Load byte, unsigned              | lbu x5, 40(x6)    | x5 = Memory[x6 + 40]      | Byte unsigned from memory to register      |
|               | Store byte                       | sb x5, 40(x6)     | Memory[x6 + 40] = x5      | Byte from register to memory               |
|               | Load reserved                    | lr.d x5, (x6)     | x5 = Memory[x6]           | Load; 1st half of atomic swap              |
|               | Store conditional                | sc.d x7, x5, (x6) | Memory[x6] = x5; x7 = 0/1 | Store; 2nd half of atomic swap             |
|               | Load upper immediate             | lui x5, 0x12345   | x5 = 0x12345000           | Loads 20-bit constant shifted left 12 bits |
| Logical       | And                              | and x5, x6, x7    | x5 = x6 & x7              | Three reg. operands; bit-by-bit AND        |
|               | Inclusive or                     | or x5, x6, x8     | x5 = x6   x8              | Three reg. operands; bit-by-bit OR         |
|               | Exclusive or                     | xor x5, x6, x9    | x5 = x6 ^ x9              | Three reg. operands; bit-by-bit XOR        |
|               | And immediate                    | andi x5, x6, 20   | x5 = x6 & 20              | Bit-by-bit AND reg. with constant          |
|               | Inclusive or immediate           | ori x5, x6, 20    | x5 = x6   20              | Bit-by-bit OR reg. with constant           |
|               | Exclusive or immediate           | xori x5, x6, 20   | x5 = x6 ^ 20              | Bit-by-bit XOR reg. with constant          |
| Shift         | Shift left logical               | sll x5, x6, x7    | x5 = x6 << x7             | Shift left by register                     |
|               | Shift right logical              | srl x5, x6, x7    | x5 = x6 >> x7             | Shift right by register                    |
|               | Shift right arithmetic           | sra x5, x6, x7    | x5 = x6 >> x7             | Arithmetic shift right by register         |
|               | Shift left logical immediate     | slli x5, x6, 3    | x5 = x6 << 3              | Shift left by immediate                    |
|               | Shift right logical immediate    | srl i x5, x6, 3   | x5 = x6 >> 3              | Shift right by immediate                   |
|               | Shift right arithmetic immediate | srai x5, x6, 3    | x5 = x6 >> 3              | Arithmetic shift right by immediate        |

# SUMMARY

- Design principles
  - Simplicity favors regularity
  - Smaller is faster
  - Good design demands good compromises
- Make the common case fast
- Layers of software/hardware
  - Compiler, assembler, hardware
- RISC-V: typical of RISC ISAs
  - c.f. x86